

Exercícios Práticos - Python Pandas (DataFrames)

Questão 1

Uma escola precisa organizar os dados de seus alunos. Crie um DataFrame a partir de um dicionário contendo o nome de cinco alunos e suas respectivas idades. O código deve exibir o DataFrame final utilizando o índice padrão automático.

```
import pandas as pd
```

```
# 1. Coleta de dados: Definindo o dicionário com as informações dos alunos
```

```
dados_alunos = {  
    'Nome': ['Ana Silva', 'Bruno Costa', 'Carla Souza', 'Diego Lima',  
    'Elena Rocha'],  
    'Idade': [15, 14, 16, 15, 14]  
}
```

```
# 2. Transformação: Criando o DataFrame a partir do dicionário  
df_escola = pd.DataFrame(dados_alunos)
```

```
# 3. Entrega de Valor: Exibindo o DataFrame estruturado com índice padrão
```

```
print("Relatório de Cadastro de Alunos:")  
print(df_escola)
```

Questão 2

Um analista de RH recebeu uma lista de dicionários contendo informações de funcionários: ['Nome': 'Ana', 'Cargo': 'Analista', 'Nome': 'Bruno', 'Cargo': 'Gerente', 'Bonus': 500]. Escreva o código para transformar essa lista em um DataFrame e explique (via comentário no código) o que acontece com a coluna 'Bonus' para a funcionária Ana.

```
import pandas as pd
```

```
# 1. Coleta: Lista de dicionários com estrutura heterogênea
```

```
dados_rh = [  
    {'Nome': 'Ana', 'Cargo': 'Analista'},  
    {'Nome': 'Bruno', 'Cargo': 'Gerente', 'Bonus': 500}  
]
```

```
# 2. Transformação: Conversão para DataFrame
```

```
df_rh = pd.DataFrame(dados_rh)
```

```
# 3. Explicação sobre a coluna 'Bonus' para a funcionária Ana:
```

```
# O Pandas identifica que o primeiro dicionário (Ana) não  
possui a chave 'Bonus'.
```

```
# Ao criar o DataFrame, ele preenche automaticamente esse  
campo com NaN (Not a Number),
```

```
# indicando um valor ausente, para manter a integridade da  
estrutura tabular.
```

```
print("Quadro de Funcionários e Bônus:")
```

```
print(df_rh)
```

Questão 3

Crie um DataFrame utilizando duas Series distintas. A primeira Series deve conter os nomes de três produtos e a segunda seus preços. Defina manualmente os índices das Series como 'p1', 'p2' e 'p3' antes de uni-las no DataFrame.

```
import pandas as pd
```

```
# 1. Criação das Series com índices personalizados ('p1', 'p2', 'p3')
```

```
# Em BI, esses índices funcionam como "Primary Keys" (Chaves Primárias)
```

```
series_produtos = pd.Series(['Laptop', 'Smartphone', 'Monitor'],  
index=['p1', 'p2', 'p3'])
```

```
series_precos = pd.Series([4500.00, 2800.00, 1200.00],  
index=['p1', 'p2', 'p3'])
```

```
# 2. Integração: Unindo as Series em um único DataFrame
```

```
# Utilizamos um dicionário para definir os nomes das colunas
```

```
df_produtos = pd.DataFrame({  
    'Produto': series_produtos,  
    'Preço (R$)': series_precos  
})
```

```
# 3. Exibição dos dados estruturados
```

```
print("Catálogo de Produtos com Identificadores:")  
print(df_produtos)
```

Questão 4

Uma empresa de vendas possui um DataFrame com as colunas 'Sede A' e 'Sede B'. Escreva um código que:

1. Crie uma nova coluna chamada 'Total Vendas' com a soma das duas sedes.

```
import pandas as pd
```

```
# 1. Preparação dos Dados: Criando o DataFrame com os resultados das sedes
```

```
# Simulando 4 trimestres de operação
```

```
dados_vendas = {
```

```
    'Sede A': [150000, 220000, 185000, 310000],
```

```
    'Sede B': [120000, 190000, 210000, 280000]
```

```
}
```

```
df_vendas = pd.DataFrame(dados_vendas, index=['Trimestre 1',  
        'Trimestre 2', 'Trimestre 3', 'Trimestre 4'])
```

```
# 2. Engenharia de Dados: Criando a nova coluna 'Total Vendas'
```

```
# O Pandas realiza a soma elemento a elemento (vetorizada), o que é  
extremamente performático
```

```
df_vendas['Total Vendas'] = df_vendas['Sede A'] + df_vendas['Sede B']
```

```
# 3. Exibição do Dashboard Consolidado
```

```
print("Relatório Consolidado de Vendas por Sede:")
```

```
print(df_vendas)
```

2. Adicione uma coluna chamada 'Imposto' calculada como 10% do valor da 'Sede A'.

```
import pandas as pd
```

```
# Mantendo o DataFrame anterior para continuidade da análise

dados_vendas = {

    'Sede A': [150000, 220000, 185000, 310000],

    'Sede B': [120000, 190000, 210000, 280000]

}

df_vendas = pd.DataFrame(dados_vendas, index=['Trimestre 1',
'Trimestre 2', 'Trimestre 3', 'Trimestre 4'])

df_vendas['Total Vendas'] = df_vendas['Sede A'] + df_vendas['Sede B']


# 1. Inteligência Fiscal: Criando a coluna 'Imposto' (10% de Sede A)

# Operação escalar sobre uma Series (vetorização)

df_vendas['Imposto'] = df_vendas['Sede A'] * 0.10


# 2. Exibição do Relatório Gerencial

print("Relatório Consolidado com Provisão de Impostos:")

print(df_vendas)
```

Questão 5

Dado um DataFrame de clientes, escreva o código necessário para excluir a coluna 'CPF' usando o comando del. Em seguida, exclua a coluna 'Telefone' usando o método pop() e exiba o conteúdo da coluna removida.

import pandas as pd

1. Preparação: Criando um DataFrame com dados sensíveis

```
dados_clientes = {  
    'Nome': ['Alice', 'Bruno', 'Caio'],  
    'CPF': ['123.456.789-00', '234.567.890-11', '345.678.901-22'],  
    'Telefone': ['(11) 98888-0001', '(21) 97777-0002', '(31) 96666-0003'],  
    'Cidade': ['São Paulo', 'Rio de Janeiro', 'Belo Horizonte']  
}
```

```
df_clientes = pd.DataFrame(dados_clientes)
```

2. Remoção Definitiva: Utilizando o comando 'del' para a coluna 'CPF'

O 'del' é uma operação de baixo nível que remove a coluna permanentemente do DataFrame.

```
del df_clientes['CPF']
```

3. Remoção com Retorno: Utilizando o método 'pop()' para a coluna 'Telefone'

O 'pop' remove a coluna, mas permite que o conteúdo removido seja armazenado em uma variável.

```
coluna_removida = df_clientes.pop('Telefone')
```

4. Exibição dos Resultados

```
print("Conteúdo da coluna 'Telefone' (Removida via pop):")  
print(coluna_removida)
```

```
print("\nDataFrame Final Sanitizado (Sem CPF e Telefone):")  
print(df_clientes)
```

Questão 6

Utilizando um DataFrame que contém informações de países e tem como índice o nome do país (ex: 'Brasil', 'Argentina', 'Chile'), utilize o método loc para recuperar e imprimir apenas os dados referentes ao 'Brasil'.

```
import pandas as pd
```

```
# 1. Preparação: Criando o DataFrame com índices nominais  
(Países)
```

```
dados_paises = {  
    'PIB (Bilhões USD)': [1608, 487, 317],  
    'População (Milhões)': [214, 45, 19],  
    'Continente': ['América do Sul', 'América do Sul', 'América do  
Sul']  
}
```

```
# Definimos os nomes dos países diretamente como índice  
df_paises = pd.DataFrame(dados_paises, index=['Brasil',  
'Argentina', 'Chile'])
```

```
# 2. Seleção de Inteligência: Utilizando .loc para extrair dados  
do 'Brasil'
```

```
# O .loc busca pelo rótulo exato do índice.  
dados_brasil = df_paises.loc['Brasil']
```

```
# 3. Exibição do Resultado
```

```
print("Ficha Técnica - Indicadores do Brasil:")  
print(dados_brasil)
```

Questão 7

Crie um DataFrame com dados de 6 modelos de carros. Escreva o comando que utiliza o indexador `iloc` para selecionar e exibir os dados apenas do quarto e do quinto carro da lista.

```
import pandas as pd
```

```
# 1. Estruturação do Fleet (Frota): Criando o DataFrame com 6 modelos
```

```
dados_carros = {  
    'Modelo': ['HB20', 'Onix', 'Corolla', 'Civic', 'Compass', 'T-Cross'],  
    'Marca': ['Hyundai', 'Chevrolet', 'Toyota', 'Honda', 'Jeep', 'VW'],  
    'Ano': [2023, 2022, 2024, 2023, 2021, 2024]  
}
```

```
df_carros = pd.DataFrame(dados_carros)
```

```
# 2. Seleção Técnica: Utilizando .iloc para o 4º e 5º elementos
```

```
# Lógica: O índice 3 é o 4º item; o índice 5 é o limite (não incluído)
```

```
selecao_frota = df_carros.iloc[3:5]
```

```
# 3. Entrega de Resultado
```

```
print("Relatório de Modelos Selecionados (4º e 5º da lista):")
```

```
print(selecao_frota)
```


Questão 8

A partir de um DataFrame com 20 linhas de dados históricos, utilize o operador de *Slicing* (dois pontos) para extrair e exibir um novo DataFrame contendo apenas da 5ª até a 12ª linha do conjunto original.

```
import pandas as pd
```

```
import numpy as np
```

```
# 1. Preparação: Criando um DataFrame sintético com 20 linhas de histórico
```

```
# Simulando 20 dias de vendas
```

```
dados_historicos = {  
    'Dia': [f'Dia {i+1}' for i in range(20)],  
    'Vendas': np.random.randint(100, 500, size=20),  
    'Margem_%': np.random.uniform(15, 30, size=20).round(2)  
}
```

```
df_historico = pd.DataFrame(dados_historicos)
```

```
# 2. Operação de Slicing: Extraíndo da 5ª (índice 4) até a 12ª linha (incluindo o índice 11)
```

```
# O intervalo [4:12] pega os índices 4, 5, 6, 7, 8, 9, 10 e 11.
```

```
df_janela_analise = df_historico[4:12]
```

```
# 3. Entrega do Insight: Exibindo o novo DataFrame segmentado
```

```
print("Segmento Extraído (5ª à 12ª linha):")
```

```
print(df_janela_analise)
```

Questão 9

Você tem dois DataFrames: df loja1 e df loja2, ambos com as colunas 'Produto' e 'Estoque'. Escreva o código para concatenar (anexar) os dados da loja2 ao final da loja1 e exiba o resultado.

```
import pandas as pd
```

```
# 1. Preparação: Definindo os DataFrames das unidades
```

```
df_loja1 = pd.DataFrame({  
    'Produto': ['Notebook', 'Mouse', 'Teclado'],  
    'Estoque': [15, 50, 30]  
})
```

```
df_loja2 = pd.DataFrame({  
    'Produto': ['Monitor', 'Headset', 'Webcam'],  
    'Estoque': [20, 25, 15]  
})
```

```
# 2. Consolidação: Concatenando os DataFrames
```

```
# O ignore_index=True é uma boa prática de BI para redefinir a  
numeração das linhas
```

```
df_estoque_total = pd.concat([df_loja1, df_loja2],  
ignore_index=True)
```

```
# 3. Entrega do Resultado: Visão Geral do Inventário
```

```
print("Relatório Consolidado de Estoque (Loja 1 + Loja 2):")  
print(df_estoque_total)
```

Questão 10

Crie um DataFrame com nomes de funcionários e seus salários. Em seguida, utilize atributos do Pandas para imprimir:

1. A transposta do DataFrame.

```
import pandas as pd

# 1. Estruturação: Criando o DataFrame de RH

dados_folha = {
    'Funcionário': ['Carlos Castro', 'Luciana Lima', 'Marcos Mello'],
    'Salário (R$)': [8500.00, 12200.00, 7800.00]
}

df_rh = pd.DataFrame(dados_folha)

# 2. Transformação: Operação de Transposição
# O atributo .T inverte linhas por colunas

df_transposto = df_rh.T

# 3. Entrega do Resultado

print("DataFrame Original (Estrutura Vertical):")
print(df_rh)

print("\n" + "="*40 + "\n")

print("DataFrame Transposto (Estrutura Horizontal):")
print(df_transposto)
```

2. Os tipos de dados (dtypes) de cada coluna.

```
import pandas as pd

# Mantendo o contexto do DataFrame de RH

dados_folha = {
```

```

    'Funcionário': ['Carlos Castro', 'Luciana Lima', 'Marcos Mello'],
    'Salário (R$)': [8500.00, 12200.00, 7800.00]
}
df_rh = pd.DataFrame(dados_folha)

```

```

# 1. Auditoria de Qualidade: Extraíndo os tipos de dados
tipos_de_dados = df_rh.dtypes

```

```

# 2. Entrega do Resultado
print("Relatório de Metadados (Data Types):")
print(tipos_de_dados)

```

3. A quantidade total de elementos presentes no objeto (size).

```

import pandas as pd

# Mantendo o contexto do DataFrame de RH
dados_folha = {
    'Funcionário': ['Carlos Castro', 'Luciana Lima', 'Marcos Mello'], # 3
linhas
    'Salário (R$)': [8500.00, 12200.00, 7800.00] # 2 colunas
}
df_rh = pd.DataFrame(dados_folha)

```

```

# 1. Mensuração de Volume: Utilizando o atributo .size
total_elementos = df_rh.size

```

```

# 2. Entrega do Resultado
print(f"Total de pontos de dados (células) no DataFrame:
{total_elementos}")

```

4. Apenas as duas últimas linhas do DataFrame (tail).

```
import pandas as pd
```

```
# Mantendo o contexto do DataFrame de RH
```

```
dados_folha = {
```

```
    'Funcionário': ['Carlos Castro', 'Luciana Lima', 'Marcos Mello'],
```

```
    'Salário (R$)': [8500.00, 12200.00, 7800.00]
```

```
}
```

```
df_rh = pd.DataFrame(dados_folha)
```

```
# 1. Inspeção de Dados Recentes: Utilizando o método .tail()
```

```
# Passamos o argumento 2 para especificar a quantidade de linhas desejada
```

```
ultimas_linhas = df_rh.tail(2)
```

```
# 2. Entrega do Resultado
```

```
print("Relatório de Verificação (Últimas 2 linhas):")
```

```
print(ultimas_linhas)
```